

Architettura degli Elaboratori

La cache – introduzione



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo
Masi, Claudio di Ciccio

[S&PdC]

5.1, 5.3 – 5.4



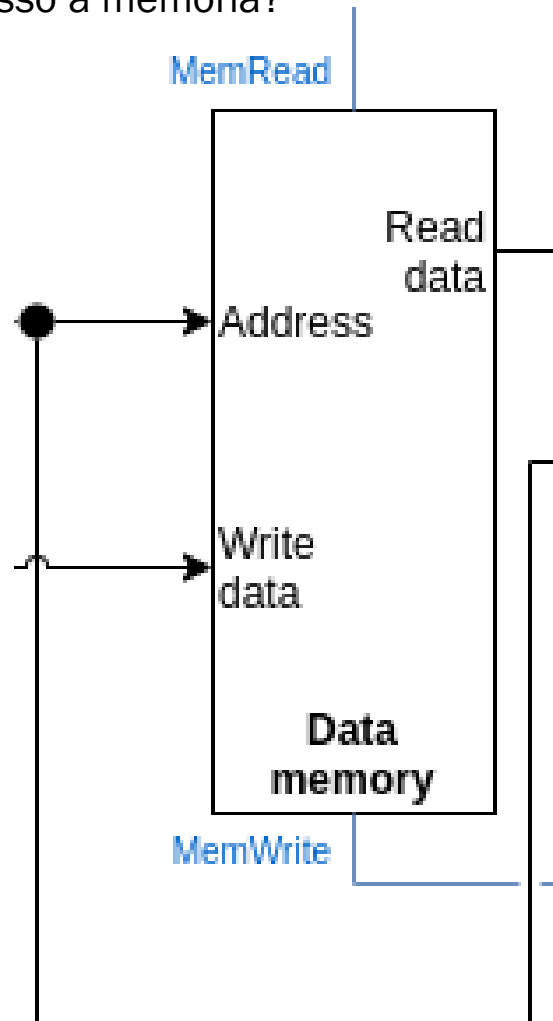
Caching della memoria principale



SAPIENZA
UNIVERSITÀ DI ROMA

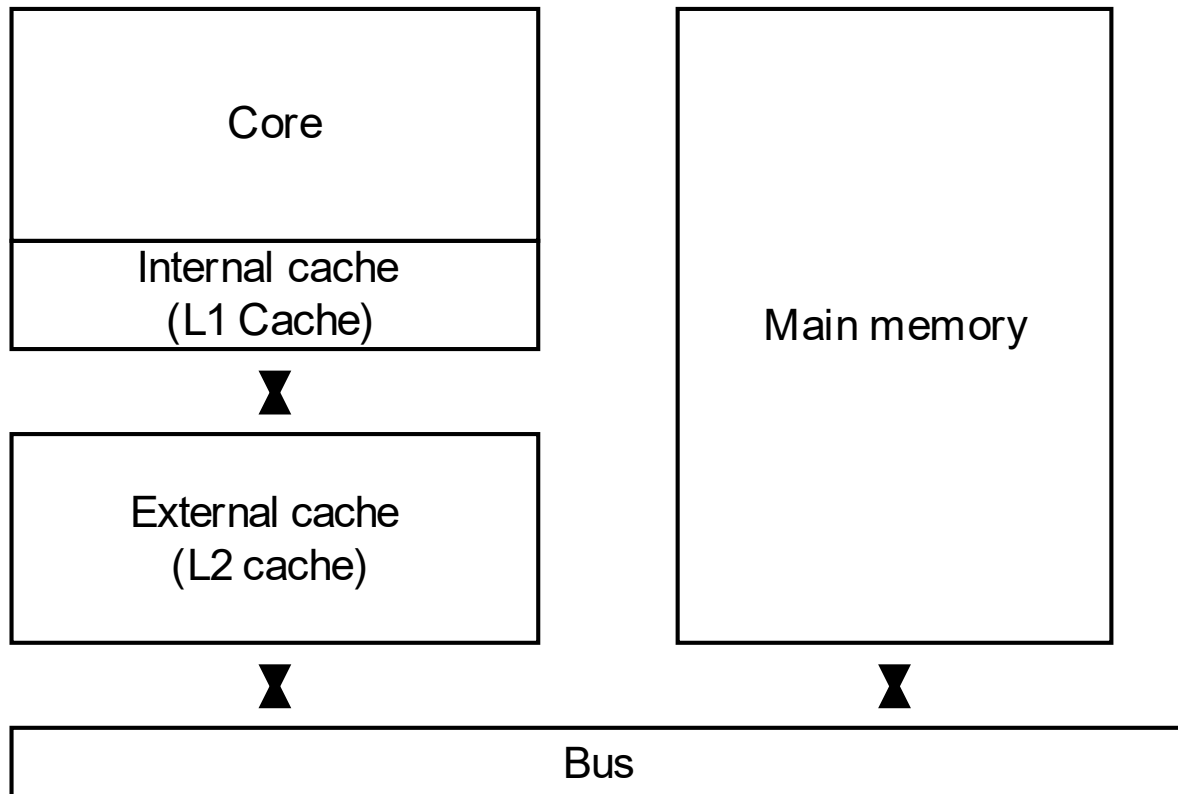
Argomento

Cosa accade in fase di accesso a memoria?

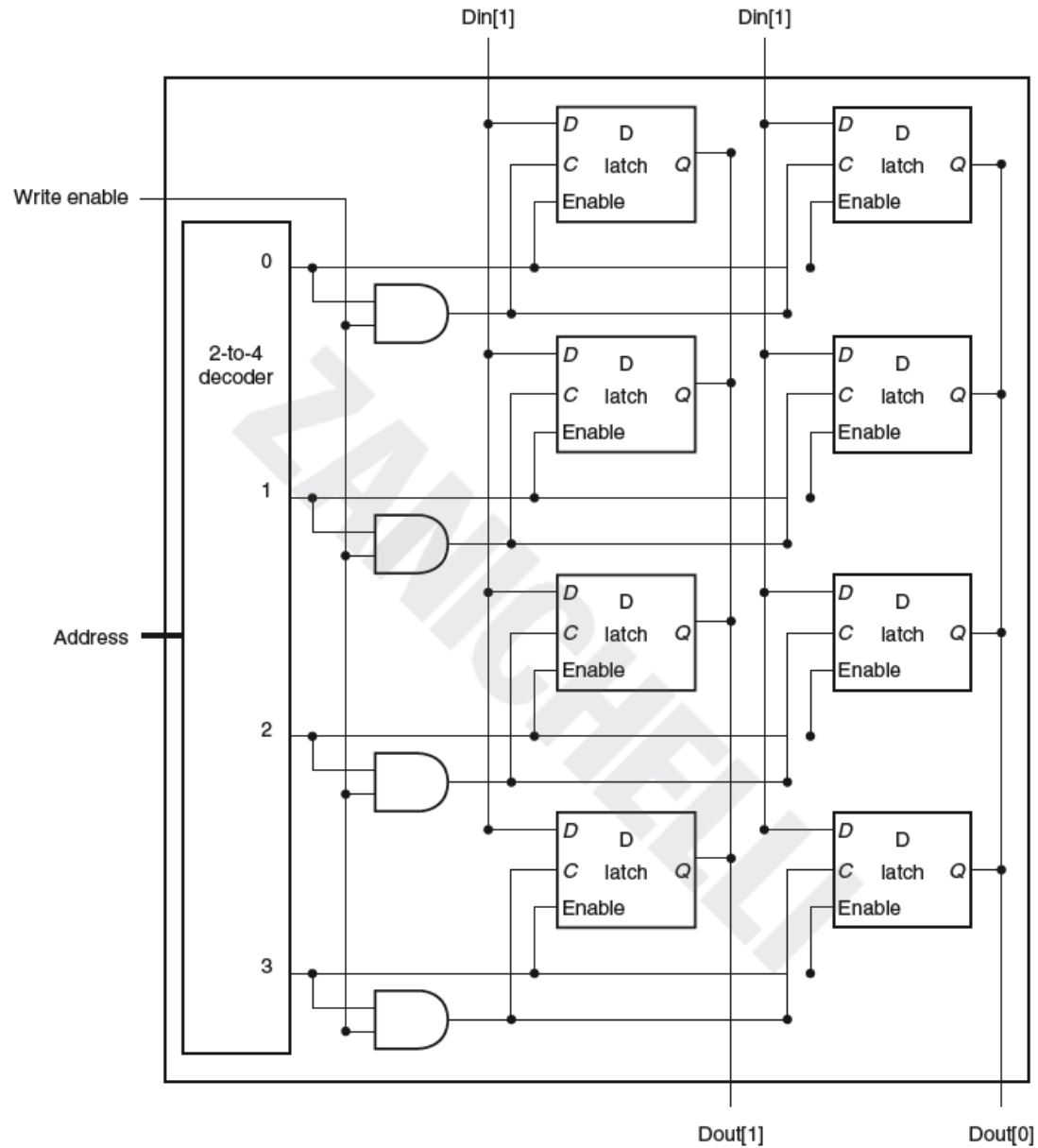


Argomento

Cosa accade in fase di accesso a memoria?



SRAM



SRAM architecture

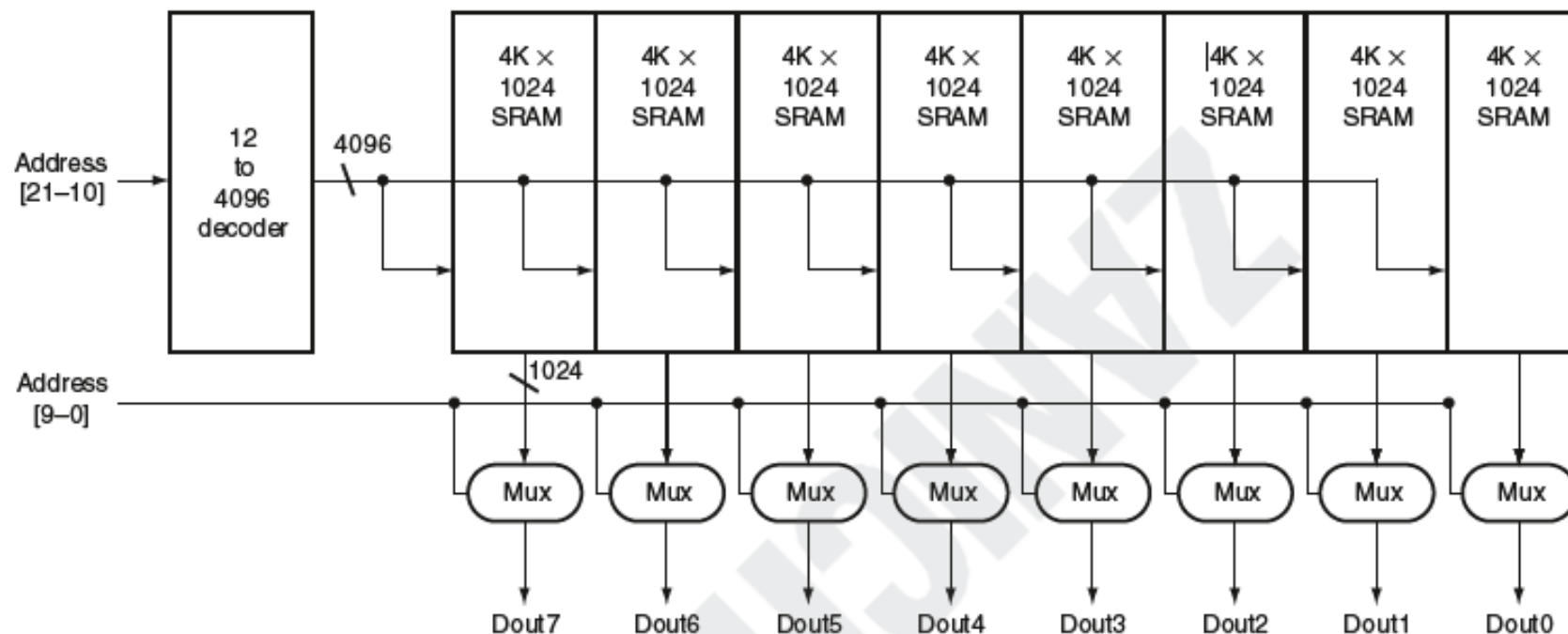
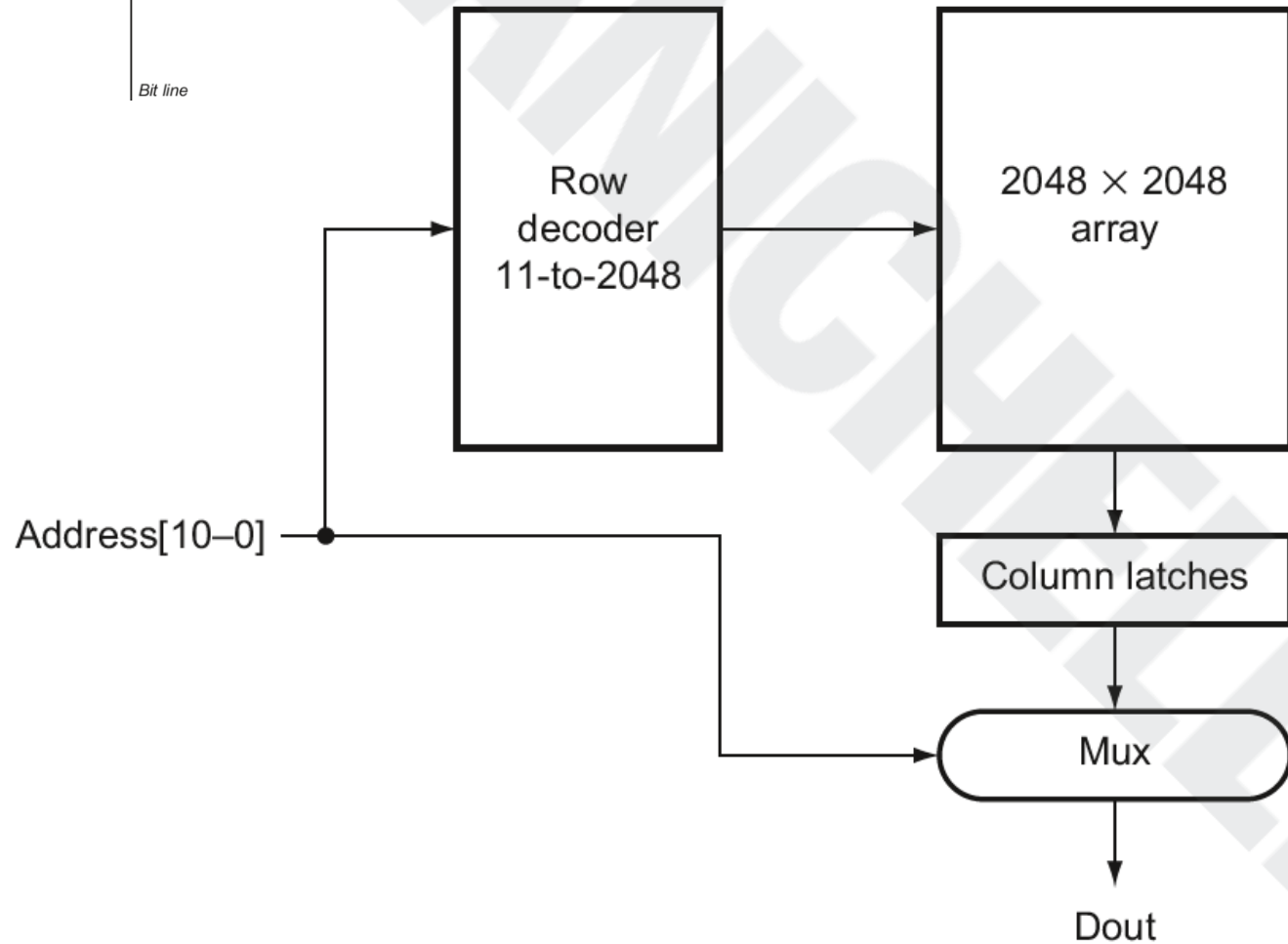
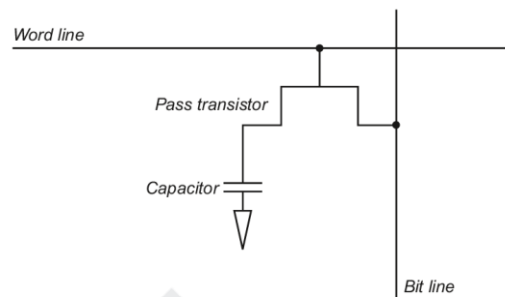


FIGURE A.9.4 Typical organization of a 4M x 8 SRAM as an array of 4K x 1024 arrays. The first decoder generates the addresses for eight 4K x 1024 arrays; then a set of multiplexors is used to select 1 bit from each 1024-bit-wide array. This is a much easier design than a single-level decode that would need either an enormous decoder or a gigantic multiplexor. In practice, a modern SRAM of this size would probably use an even larger number of blocks, each somewhat smaller.

DRAM



DRAM architecture

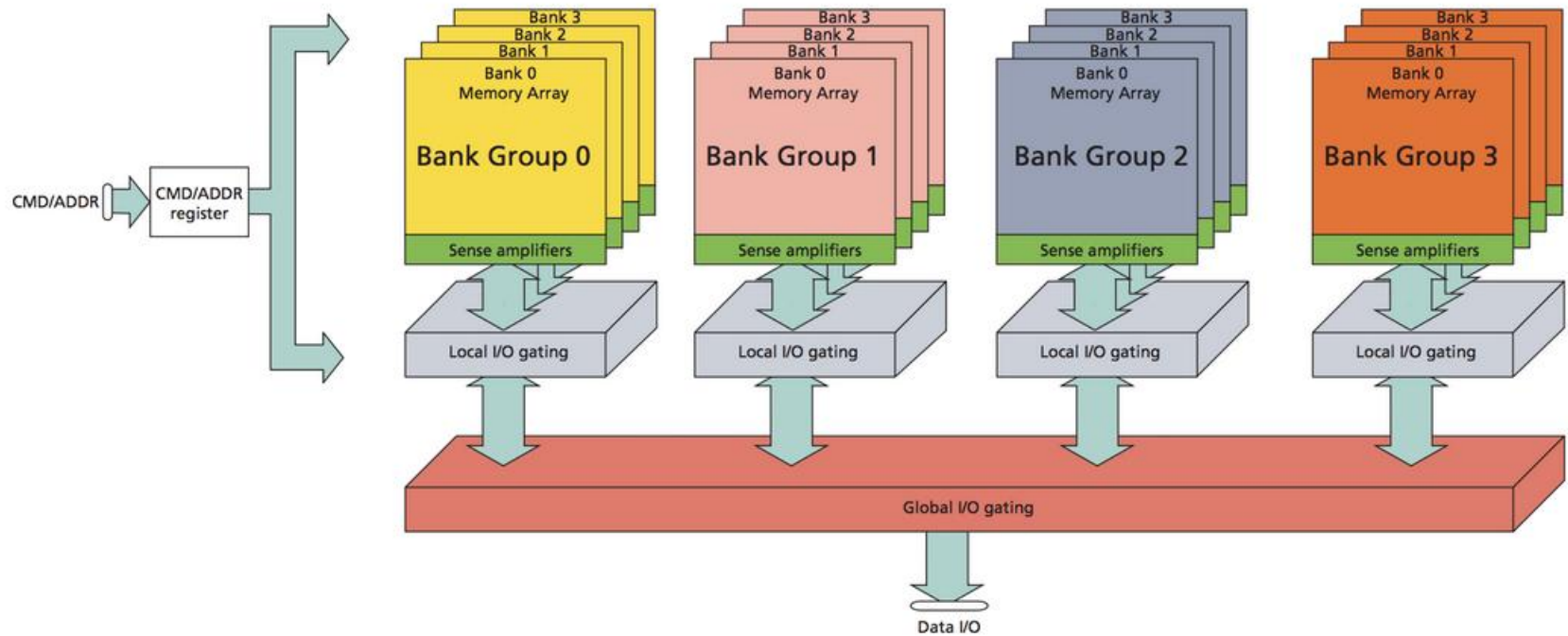


Figure 2: BankGroup & Bank (Source: Micron Datasheet)

Velocità RAM

	DRAM	DDR	DDR2	DDR3	DDR4	DDR5
Prelettura	1 - bit	2 - bit	4 - bit	8 - bit	Bit per banco	16 - bit
Velocità di trasferimento dati (MT/s)	100 - 166	266 - 400	533 - 800	1066 - 1600	2133 - 5100	3200 - 6400
Velocità di trasferimento in GB/s	0,8 - 1,3	2,1 - 3,2	4,2 - 6,4	8,5 - 14,9	17 - 25,6	38.4 - 51.2
Tensione (V)	3,3	2,5 - 2,6	1,8	1,35 - 1,5	1,2	1,1

Prestazione delle memorie

Memory	Size	Latency	Bandwidth
L1 cache	32 KB	1 nanosecond	1 TB/second
L2 cache	256 KB	4 nanoseconds	1 TB/second Sometimes shared by two cores
L3 cache	8 MB or more	10x slower than L2	>400 GB/second
MCDRAM		2x slower than L3	400 GB/second
Main memory on DDR DIMMs	4 GB-1 TB	Similar to MCDRAM	100 GB/second
Main memory on Cornelis* Omni-Path Fabric	Limited only by cost	Depends on distance	Depends on distance and hardware
I/O devices on memory bus	6 TB	100x-1000x slower than memory	25 GB/second
I/O devices on PCIe bus	Limited only by cost	From less than milliseconds to minutes	GB-TB/hour Depends on distance and hardware

Argomento

Cosa accade in fase di accesso a memoria?

Problema:

la RAM è **molto più lenta** del processore
(circa 10-100 volte più lenta)

Una memoria più veloce è **molto più costosa**

1. Principio di località temporale

- «un programma tende ad accedere allo stesso elemento in momenti vicini tra loro»

2. Principio di località spaziale

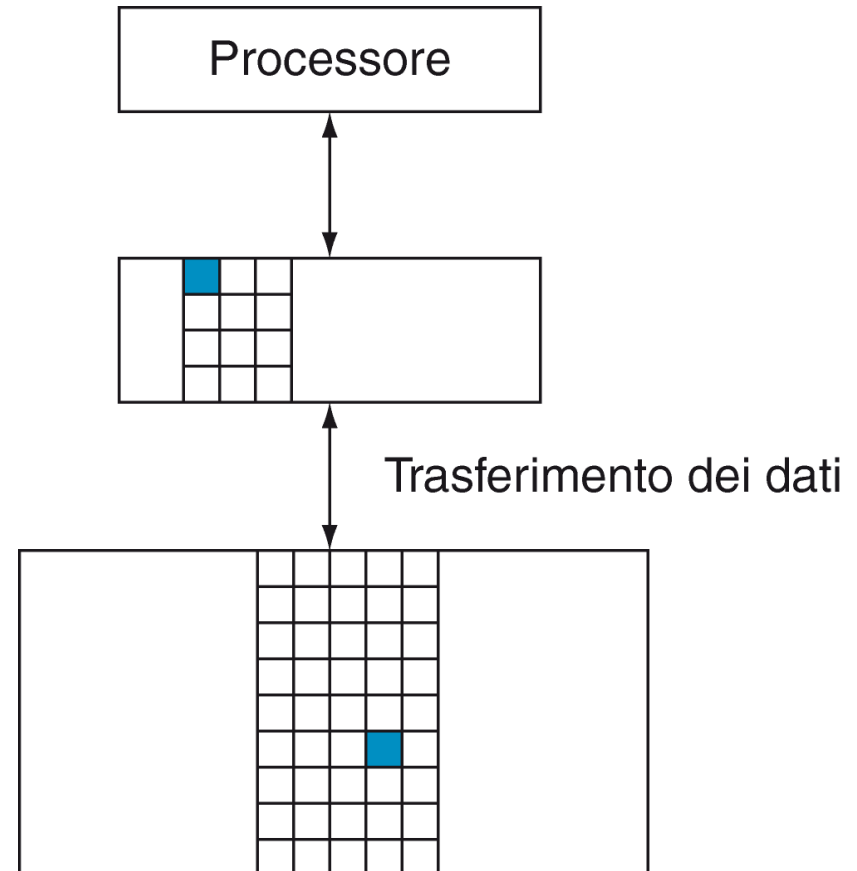
- «un programma tende ad accedere successivamente elementi di memoria vicini tra loro»

• Idea 1:

- mettiamo da parte in una **piccola** memoria **veloce** (cache) solo i **dati più usati**

• Idea 2:

- quando memorizziamo un dato **mettiamo** da parte anche i dati vicini



Argomento

Problema:

la RAM è **molto più lenta** del processore
(circa 10-100 volte più lenta)

Una memoria più veloce è **molto più costosa**

1. Principio di località temporale

- «un programma tende ad accedere allo stesso elemento in momenti vicini tra loro»

2. Principio di località spaziale

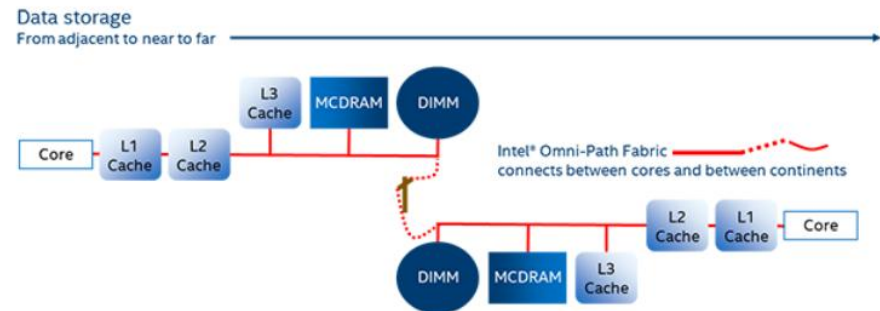
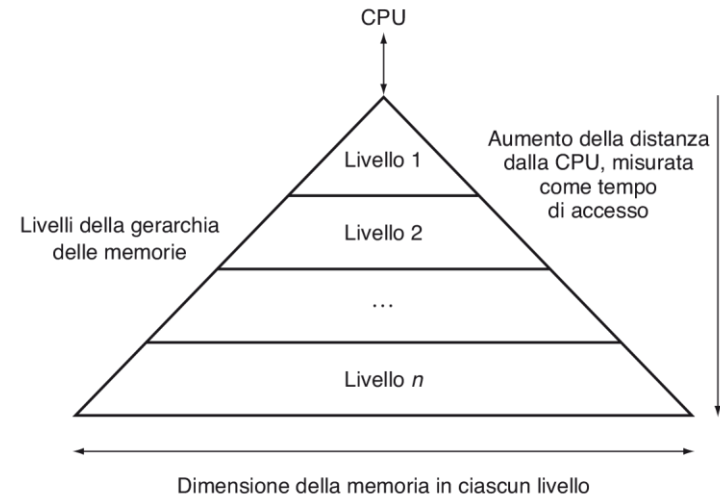
- «un programma tende ad accedere successivamente elementi di memoria vicini tra loro»

• Idea 1:

- mettiamo da parte in una **piccola** memoria **veloce** (cache) solo i **dati più usati**

• Idea 2:

- quando memorizziamo un dato **mettiamo da parte anche i dati vicini**



Schema e terminologia

La memoria è divisa in **blocchi** di dimensioni uguali

La cache contiene un numero **N** di **linee**:

Quando la CPU richiede un indirizzo:

- Il blocco corrispondente è presente → il dato è caricato dalla cache (**HIT**)
- Altrimenti → dato è preso dalla RAM e il blocco è copiato in cache (**MISS**)

Le linee contengono:

- 1 bit di **validità**
 - indica se la linea contiene dati validi
- Il campo **tag**
 - distingue quale blocco della memoria è nella linea
- Il **blocco** memorizzato nella linea

Cache

# linea	V	Tag	Blocco
0	0		
1	1	0	
2	1	1	
3	0		

Blocco 0

Blocco 1

Blocco 2

Blocco 3

Blocco 4

Blocco 5

Blocco 6

Memoria

Performance

Supponiamo che un programma preveda **1 000 000** di accessi in memoria e che il tempo di accesso sia di **100 ns**

Il tempo totale che il programma impiegherà per l'accesso è $1\,000\,000 \times 100\text{ns} = \mathbf{100\ ms}$

Se si usa una **cache** con tempo di accesso **1 ns** e la percentuale di MISS è il **10%**

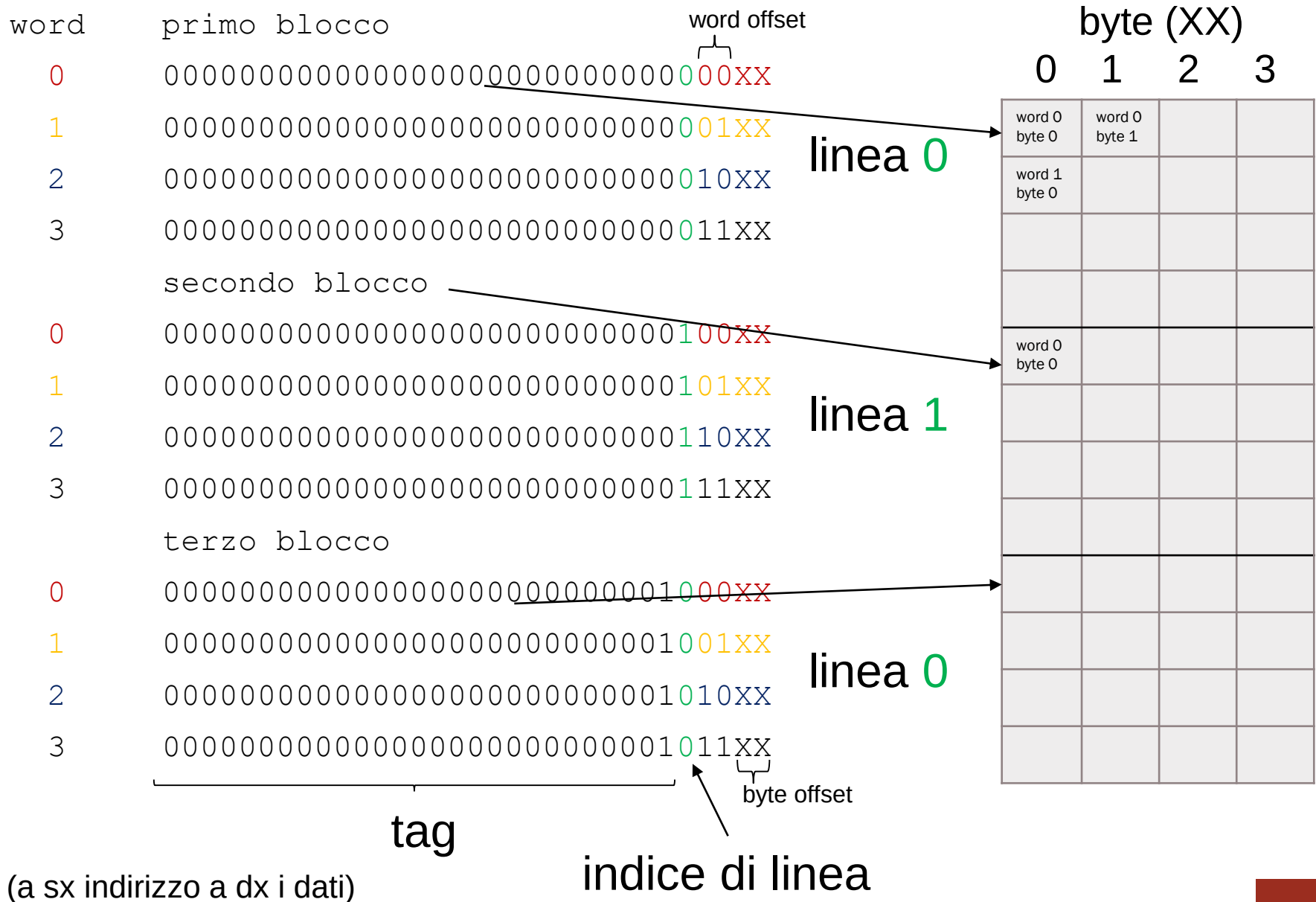
- il **90%** di **1 000 000** accessi (**HIT**) impiegheranno $1\text{ ns} \times 1\,000\,000 \times 90\% = \mathbf{0.9\ ms}$
- il **10%** rimanente (**MISS**) impiegheranno $100\text{ ns} \times 1\,000\,000 \times 10\% = \mathbf{10\ ms}$

In totale il **tempo di accesso medio** sarà $10\text{ms} + 0.9\text{ms} / 1\,000\,000 = \mathbf{10.9\ ns}$
con un aumento di velocità di $100\text{ns} / 10.9\text{ns} \cong \mathbf{9\ volte\ circa}$

Esempio: il processore FastMath

Frequenza di miss per le istruzioni	Frequenza di miss per i dati	Frequenza di miss totale
0,4%	11,4%	3,2%

Direct mapping - Blocchi di 4 word, cache di 2 linee



Direct mapping

Numero di blocco																								Offset			
Tag																								Word		Byte	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	1	0	1

Il blocco va nella linea ottenuta dal numero di blocco *modulo* il numero N di linee disponibili (ogni N blocchi lo schema si ripete)

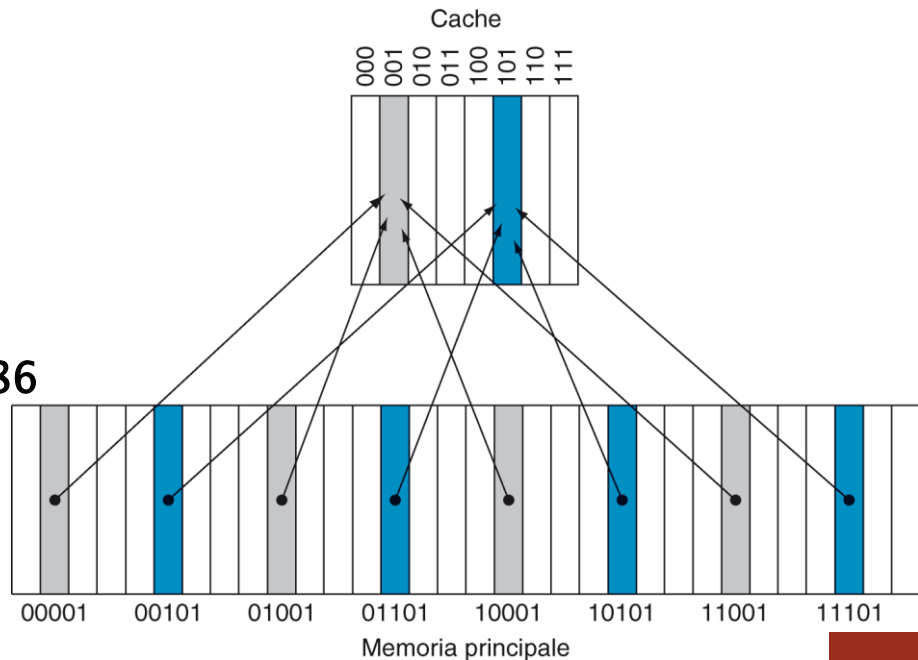
Esempio:

Direct mapped cache con:

- Blocco di 4 word = 16 byte
- 8 linee

Indirizzo: **2157 = 0x0000 086D**

- Numero di blocco = $2157 / 16 = 134 = 0x86$
- Offset di blocco = $2157 \% 16 = 13 = 0xD$
- Indice di linea = $134 \% 8 = 6 = 0x6$
- Tag = $134 / 8 = 16 = 0x10$



Esempio di sequenza di accessi

Direct mapped cache

Con 8 linee

Sequenza di accessi:

#blocco → tag indice

10110 → 10 110

11010 → 11 010

10000 → 10 000

00011 → 00 011

10010 → 10 010

10110 → 10 110

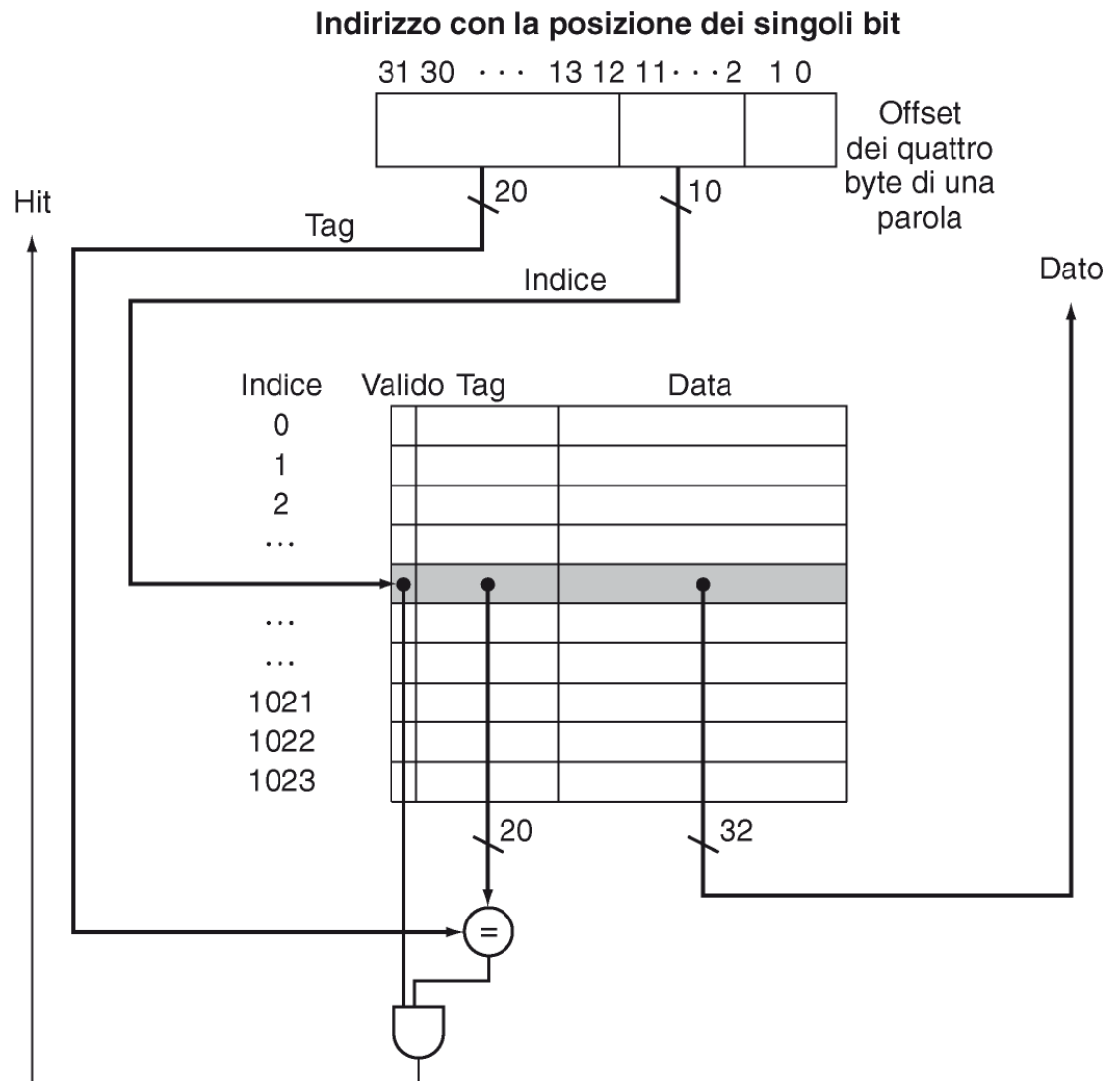
10000 → 10 000

Indice	V	Tag	Dati
000	S	10 _{due} HIT	Memoria (10000 _{due})
001	N		
010	S	10 _{due}	Memoria (10010 _{due})
011	S	00 _{due}	Memoria (00011 _{due})
100	N		
101	N		
110	S	10 _{due} HIT	Memoria (10110 _{due})
111	N		

Come determinare HIT/MISS

Cache con:

- blocchi da 1 word
- 1024 linee

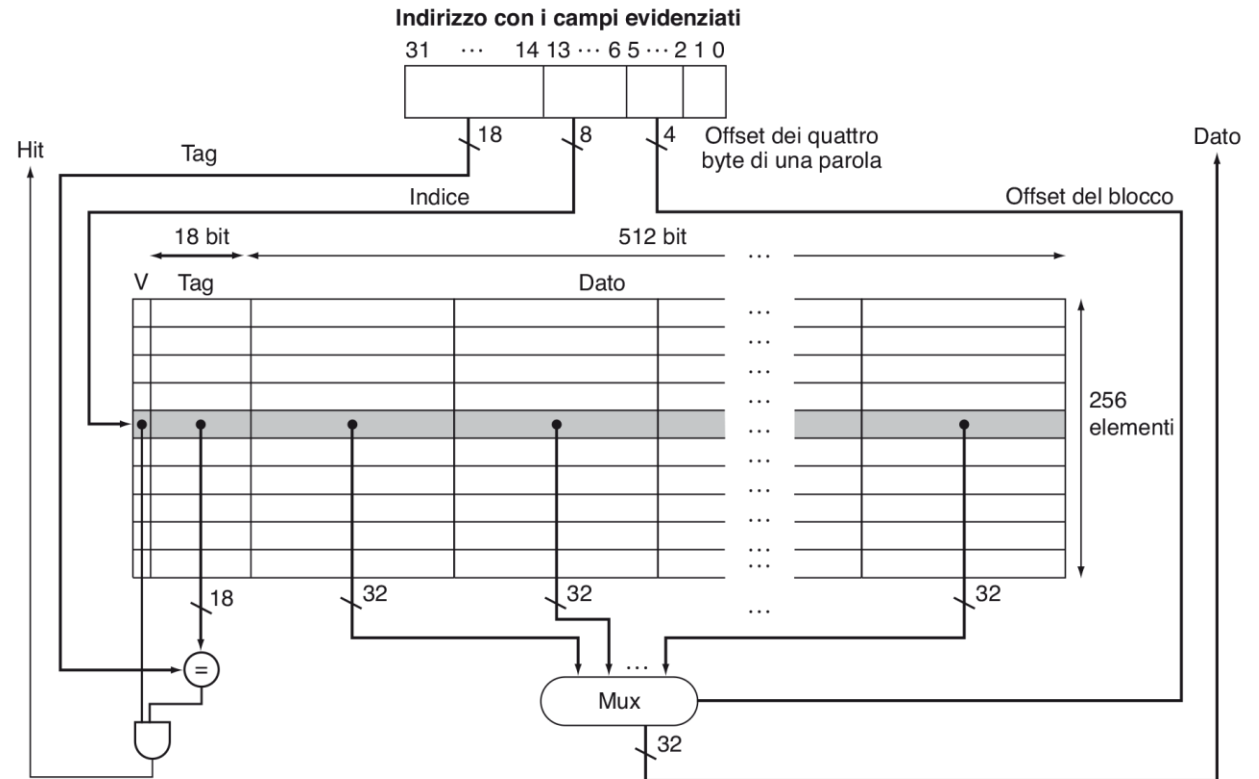


Come determinare HIT/MISS

V	Tag	Indice di linea	Offset di blocco	Offset di parola
---	-----	-----------------	------------------	------------------

Cache con:

- blocchi da 16 word
- 256 linee



Dimensioni della cache

V	Tag	Indice di linea	Offset di blocco	Offset di parola
---	-----	-----------------	------------------	------------------

Dati:

- Cache (direct mapping) con 2^n linee
- Blocchi di dimensione 2^m word da 4 byte ciascuna, cioè 2^{m+2} byte (2^{m+5} bit)
- 1 bit di validità

Dimensione del campo Tag: $32 - n - m - 2$

Dimensione della cache in bit: $2^n \times [2^m \times 32 + (32 - n - m - 2) + 1]$

Dimensioni della cache

V	Tag	Indice di linea	Offset di blocco	Offset di parola
---	-----	-----------------	------------------	------------------

Dati:

- Cache (direct mapping) con 2^n linee
- Blocchi di dimensione 2^m word da 4 byte ciascuna, cioè 2^{m+2} byte (2^{m+5} bit)
- 1 bit di validità

Dimensione del campo Tag: $32 - n - m - 2$

Dimensione della cache in bit: $2^n \times [2^m \times 32 + (32 - n - m - 2) + 1]$

Esempio:

Cache con:

- blocchi da 16 word $\rightarrow m = 4$
- 256 linee $\rightarrow n = 8$

Dimensione del campo Tag: $32 - 8 - 4 - 2 = 18$ bit

Dimensione della cache in bit: $256 \times [16 \times 32 + 18 + 1] = 2^8 \times 531 = 2^{10} \times 531 / 4$
 $135,936 \text{ kbit} = 132,75 \text{ Kibit}$